

Actividad 1. Explorando un contenedor

Objetivo: Comprender qué es un contenedor, cómo se ejecuta y cómo funciona su ciclo de vida: creación, ejecución, detención y eliminación.

Conceptos previos

- Una **imagen** es una plantilla.
- Un **contenedor** es una instancia creada a partir de una imagen.
- Un contenedor permanece activo mientras su **proceso principal** siga ejecutándose.
- El comando **docker run** crea e inicia un contenedor.
- El comando **docker exec** permite ejecutar comandos dentro de un contenedor que ya está en ejecución.

Parte 1. Contenedor interactivo

En esta parte se trabajará con un contenedor interactivo para observar su comportamiento al entrar, salir y volver a acceder a él.

1. Crear un contenedor interactivo basado en Ubuntu 22.04

```
docker run -it --name cont-int ubuntu:22.04 bash
```

En este caso, el proceso principal del contenedor es **bash**. El contenedor permanecerá activo mientras esta terminal siga abierta.

2. Explorar el entorno del contenedor

- Consultar la estructura de directorios

```
ls -la
```

- Crear un archivo dentro del contenedor

```
echo "Hola Docker" > saludo.txt
```

- Verificar que el archivo existe

```
cat saludo.txt
```

3. Salir del contenedor

```
exit
```

4. Revisar la lista de contenedores

```
docker ps -a
```

5. Preguntas de reflexión

- ¿El contenedor fue eliminado o solamente detenido?
- ¿Cómo podríamos volver a tener acceso al archivo saludo.txt?
- ¿Qué ocurriría si en lugar de reingresar al mismo contenedor ejecutamos otra vez docker run?

6. Volver a ingresar al mismo contenedor

```
docker start cont-int
docker exec -it cont-int bash
cat saludo.txt
```

Parte 2. Contenedor en segundo plano

En esta parte se trabajará con un contenedor ejecutándose en modo desacoplado.

1. Crear un contenedor en segundo plano

```
docker run -d --name cont-bg ubuntu:22.04 sleep infinity
```

Aquí el proceso principal es **sleep infinity**, por eso el contenedor permanece activo aunque no estemos dentro de él.

2. Verificar que el contenedor está en ejecución

```
docker ps
```

3. Ingresar al contenedor

```
docker exec -it cont-bg bash
```

4. Crear un archivo dentro del contenedor

```
echo "Hola Docker" > saludo.txt
cat saludo.txt
exit
```

5. Volver a entrar y comprobar persistencia

```
docker exec -it cont-bg bash
cat saludo.txt
exit
```

6. Crear otro archivo sin entrar al contenedor

```
docker exec cont-bg touch otro.txt
```

7. Verificar los archivos dentro del contenedor

```
docker exec -it cont-bg bash
ls -la
exit
```

8. Intentar borrar el contenedor

```
docker rm cont-bg
```

9. Analizar el problema

- ¿Por qué no fue posible borrar el contenedor?
- ¿Qué condición exige Docker para eliminar un contenedor con `docker rm`?
- ¿Qué diferencia hay entre un contenedor detenido y uno en ejecución?

10. Solucionar el problema

Opción 1: detener y luego borrar

```
docker stop cont-bg
docker rm cont-bg
```

Opción 2: forzar la eliminación

```
docker rm -f cont-bg
```

Preguntas de cierre

- 1 ¿Cuál es la diferencia entre una imagen y un contenedor?
- 2 ¿Qué hace el comando docker run?
- 3 ¿Por qué el contenedor de la Parte 1 se detiene cuando se ejecuta exit?
- 4 ¿Por qué el archivo saludo.txt sigue existiendo al volver a entrar al mismo contenedor?
- 5 ¿Por qué ese archivo no aparecería en un contenedor nuevo creado desde la misma imagen?
- 6 ¿Qué diferencia hay entre docker run y docker exec?
- 7 ¿Por qué docker rm cont-bg genera un error mientras el contenedor sigue en ejecución?
- 8 ¿Qué papel cumple el proceso principal en el ciclo de vida del contenedor?

Resultados esperados

- Identificar la diferencia entre imagen y contenedor.
- Crear contenedores en modo interactivo y en segundo plano.
- Entrar y salir de un contenedor correctamente.
- Verificar el estado de los contenedores con docker ps y docker ps -a.
- Comprender que los cambios hechos dentro del mismo contenedor persisten mientras este no sea eliminado.
- Distinguir entre run, exec, stop, start y rm.